AIU. ALAMEIN INTERNATIONAL UNIVERSITY
جامعة العلمين الدولية

NN SEMESTER 5 PROJECT

Neural Networks AIE231
Fall 24/25

Ramez Ezzat
22100506

Rana Hossam
22101478

Alaaeldin Ibrahim
22101463

Ahmed Fathy
22101981

Project and Problem Description

Welcome to the year 2912, where your data science skills are needed to solve a cosmic mystery. We've received a transmission from four lightyears away and things aren't looking good.

The Spaceship Titanic was an interstellar passenger liner launched a month ago. With almost 13,000 passengers on board, the vessel set out on its maiden voyage transporting emigrants from our solar system to three newly habitable exoplanets orbiting nearby stars. While rounding Alpha Centauri en route to its first destination—the torrid 55 Cancri E—the unwary Spaceship Titanic **collided with a spacetime anomaly hidden within a dust cloud.** Sadly, it met a similar fate as its namesake from 1000 years before. Though the ship stayed intact, **almost half of the passengers were transported to an alternate dimension!**

Our goal is to predict wither the passengers were transported to an alternate dimension

competition link

Our Methodology

Step 1: filter the data

- We reduced the data from 29 columns to 8 columns, removed the unnecessary or irrelevant features to optimize the model's performance.

Step 2: pre-process the data

- Normalized the data so we're not dealing with a big difference in data values.
- Got rid of the missing data by filling it with the local mean.
- Got rid of duplicate data.
- Got rid of outliers.
- Corrected errors in the data.

Step 3: plan the neural network's initial architecture

- 8 input streams - 3 hidden layers - 1 output: true, false

Step 4: initialize hyperparameters

- Learning rate is set to 0.001
- Number of neurons per layer is set to 128
- Batch size is set to 32
- ReLU activation function for hidden layers, and Sigmoid activation function for the output layer.
- Epochs is set to 1000
- Cross-Entropy loss function

Step 5: initialize weights and biases

- initialized the weights using the xavier.
- Initialize biases to 0

Step 6: optimize weights and biases

- Using the stochastic gradient descent optimization method
- Regularize the data with the drop out

Step 7: optimize hyperparameters

- We used Grid Search to optimize our hyperparameters

Step 8: train and build the model

Step 9: predict results

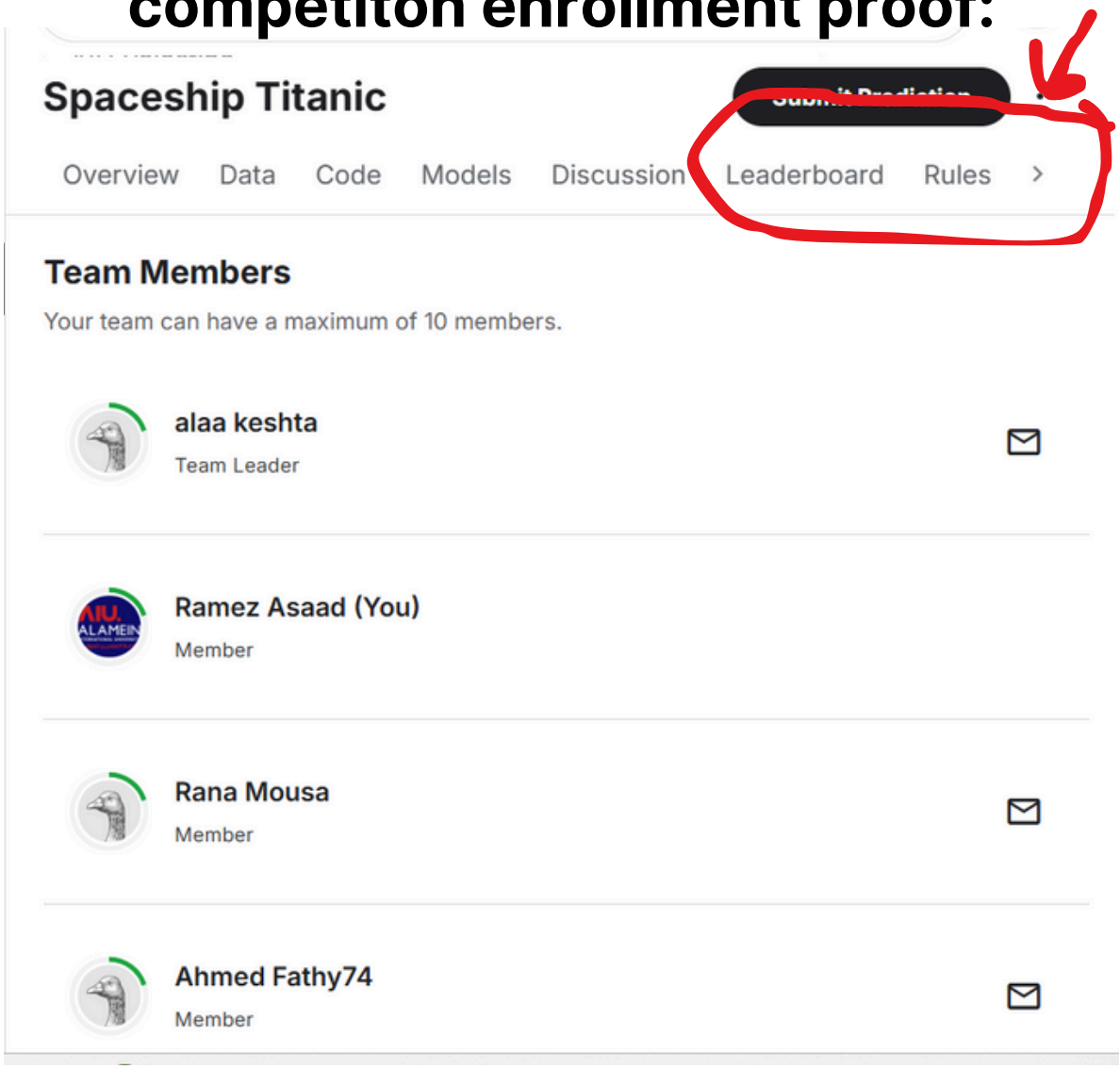
Data description

the data we’re working with is a set of personal records recovered from the ship’s damaged computer system

Data features

- **PassengerId** - A unique Id for each passenger
- **HomePlanet** - The planet the passenger departed from, typically their planet of permanent residence.
- **CryoSleep** - Indicates whether the passenger elected to be put into suspended animation for the duration of the voyage. Passengers in cryosleep are confined to their cabins.
- **Cabin** - The cabin number where the passenger is staying. Takes the form deck/num/side, where side can be either P for Port or S for Starboard.
- **Destination** - The planet the passenger will be debarking to.
- **Age** - The age of the passenger.
- **VIP** - Whether the passenger has paid for special VIP service during the voyage.
- **RoomService, FoodCourt, ShoppingMall, Spa, VRDeck** - Amount the passenger has billed at each of the Spaceship Titanic's many luxury amenities.
- **Name** - The first and last names of the passenger.
- **Transported** - Whether the passenger was transported to another dimension. This is the target, the column you are trying to predict.

competiton enrollment proof:



Our Process

we started with the data preprocessing

General Cleaning and Column Selection

Training Data:
Selected columns: ['PassengerId', 'CryoSleep', 'Cabin', 'Destination', 'Age', 'VIP', 'Transported'].
Missing values in categorical columns (CryoSleep, Cabin, Destination, VIP) filled with defaults:
CryoSleep and VIP: False.
Cabin and Destination: 'Unknown'.
Missing numerical values (Age) replaced with the column mean and converted to integers.
Test Data:
Selected columns: ['PassengerId', 'CryoSleep', 'Cabin', 'Destination', 'Age', 'VIP'].
Same imputation strategy applied as for the training data.

Feature Transformation

Converted boolean columns (CryoSleep, VIP) to integers (0 and 1).
Cabin Feature Transformation:
Split the Cabin column into two features: Cabin_Deck (deck identifier) and Cabin_Side (side of the cabin).
Mapped Cabin_Deck and Cabin_Side values to numerical representations using dictionaries.
Destination Feature Encoding:
Applied label encoding to the Destination column to convert categorical values to numeric.

Outlier Removal

Selected only numerical columns for outlier detection.
Calculated Z-scores for numerical columns.
Identified rows with any Z-score > 3 as outliers.
Dropped these outliers from the training data.

Data Splits

Split X_train from PassengerId and Transported.
Split x_test from PassengerId.
PassengerId stored separately as z_train and z_test.

Final Outputs

Created a cleaned training dataset (X_train_cleaned) with outliers removed.
Created transformed and imputed test dataset (x_test).

Phase 1: Building a model from scratch

Forward Propagation Basics:
- Implemented basic forward propagation for a single neuron with input, weight, and bias.

Activation Functions:
- Defined sigmoid for binary outputs.
- Implemented ReLU for hidden layers to add non-linearity.
- Added ReLU derivative for backpropagation.

Cost Function:
- Used binary cross-entropy to calculate the loss for classification tasks.

Weight Initialization:
- Used Xavier Initialization to initialize weights, ensuring proper gradient flow during training.

Gradient Descent Optimization:
- Calculated gradients for weights and biases using backpropagation.
- Updated weights and biases using the gradients and learning rate.

Dropout Regularization:
- Introduced a dropout function to randomly deactivate neurons during training, reducing overfitting.

Single Layer Model Training:
- Built a single-layer neural network and trained it using sigmoid activation, tracking cost over iterations.

Multi-Layer Model Construction:
- Constructed a neural network with multiple hidden layers (3 hidden layers with ReLU activation).
- Used dropout after each layer to improve generalization.

Backward Propagation for Multi-Layer Network:
- Computed gradients for each layer (weights and biases) during backpropagation.
- Updated parameters layer-by-layer starting from the output back to the input.

Training Multi-Layer Network:
- Trained the neural network on standardized input data.
- Tracked the cost after every 100 iterations to monitor progress.

Final Output:
- Returned trained weights, biases for all layers, and the cost evolution over iterations.

Key Observations:
- The model shows significant improvement in cost during the first few iterations, with the cost dropping sharply from 5.7009 to 1.8692 by iteration 300.
- After iteration 400, the cost stabilizes and converges around 0.5630, indicating the model has learned the underlying patterns but has reached a plateau.
- The final cost value of 0.5631 suggests that the model may have converged to a local minimum, with very little improvement after 900 iterations.

Conclusion:
- The model successfully reduces its error significantly, but the improvement slows down considerably after around 400 iterations.
- The final cost suggests that while the model has made substantial progress, further training or tuning might not yield significant improvements without changing hyperparameters or adding more complexity.

Phase 2: Building a model with external libraries

Gradient Descent with Logistic Regression (External Library)

Scaling Data:

- Use StandardScaler to standardize the features (input data).

Logistic Regression Model:

- Import LogisticRegression from sklearn.linear_model.
- Initialize the model with penalty=None (no regularization), solver='lbfgs' (optimized solver for small datasets), and a specified number of iterations.
- Fit the model using model.fit(x, y) where x is the feature matrix and y is the target labels.

Extract Weights and Bias:

- Use model.coef_ to get the model's weights and model.intercept_ for the bias term.

Predict Probabilities:

- Use model.predict_proba(x) to obtain the predicted probabilities for each class.

Compute Cost:

- Use log_loss() from sklearn.metrics to calculate the binary cross-entropy loss between predicted and true values.

Return Results:

- Return the weights, bias, and cost after fitting the model.

ReLU Activation for Single Layer (External Library)

Scaling Data:

- Standardize the data with StandardScaler.

Model Construction:

- Use Sequential from Keras to build the model.
- Add a single dense layer with ReLU activation using Dense(1, activation='relu').

Model Compilation:

- Compile the model with Adam optimizer and BinaryCrossentropy loss function.

Training:

- Fit the model on the training data for a specified number of epochs using model.fit().

Extract Weights and Bias:

- Use model.layers[0].get_weights() to retrieve the weights and bias from the first layer.

Print Final Cost:

- Print the final loss value from the training history.

ReLU for Multiple Layers with Dropout (External Library)

Scaling Data:

- Standardize the data using StandardScaler.

Model Construction:

- Use Sequential from Keras and add multiple layers:
- First hidden layer with ReLU activation and Dropout (0.2).
- Second hidden layer with ReLU activation and Dropout (0.2).
- Third hidden layer with ReLU activation and Dropout (0.2).
- Output layer with sigmoid activation for binary classification.

Model Compilation:

- Compile the model with Adam optimizer and BinaryCrossentropy loss function.

Training:

- Train the model using model.fit() for a set number of epochs.

Extract Weights and Bias:

- Use model.layers[i].get_weights() to extract the weights and biases for each layer (hidden layers and output layer).

Print Final Cost:

- Print the final loss value after training.

Prediction:

- Use the trained model to predict on test data by calling model.predict(x_test).
- Convert the predicted probabilities into boolean predictions (True for probability > 0.5).

Save Results:

- Create a DataFrame with PassengerID and the corresponding predictions.
- Save the predictions to a CSV file using pd.DataFrame.to_csv().

Performance

- Initial Accuracy: The model started with an accuracy of 66.14% on the first epoch.

- Final Accuracy: By the 10th epoch, the accuracy improved to 73.93%.

- Loss: The loss started at 0.6000 and decreased steadily throughout the epochs, reaching a final value of 0.5444 by the 10th epoch.

Key Observations:

- The model exhibited a consistent improvement in accuracy, though the rate of improvement slowed down after the 2nd or 3rd epoch.
- The loss showed a steady decrease, which indicates that the model is learning and minimizing the error over time.
- The final loss is moderate, suggesting that the model has made decent progress but may benefit from further tuning or additional training epochs for better performance.